

SC102 L13: 文字动画系统——详细学习笔记

涵盖：文字渲染原理 | TextMeshPro (TMP) | SDF 技术 | 访问 TMP 字符信息 | 打字机效果 | Inline 标记动画

1 Unity 文字渲染原理

[p.4-5]

文字渲染三步走：

1. **字体图集 (Font Atlas)**：导入字体文件时，Unity 生成一个图集——包含所有字符的纹理（每个字符是一个小纹理）。传统 Text 组件使用位图图集，放大后会模糊。

2. **文字网格 (Text Mesh)**：Text 组件为每个字符生成一个 Quad（矩形面片）。每个 Quad 顶点的 UV 对应图集中该字符的位置。这就是为什么叫“Text Mesh”——文字本质上是通过 Mesh+纹理渲染的。

3. **渲染**：默认 Shader 读取字体图集纹理，根据每个 Quad 的 UV 做纹理映射，显示对应字符。

为什么传统 Text 组件总是很模糊？位图图集在放大时像素不足，插值导致模糊。TextMeshPro 的 SDF 技术解决了这个问题。

2 TextMeshPro (TMP)

[p.6-7]

TMP 是 Unity Text 的替代品，使用 **SDF (Signed Distance Field, 有向距离场)** 技术。图集中每个像素存储的不是颜色值，而是**距最近字符边线的距离**——正值在字符外，负值在字符内，0 在边界上。这使得字符在任何缩放级别下都保持清晰锐利（Shader 根据距离值决定像素透明度）。

TMP 核心功能：

1. SDF 文字渲染：大小无关的清晰显示
2. 强大的图集生成工具：可自定义字符集优化
3. 每字符独立控制：可单独调整每个字符的位置/大小/颜色（文字动画系统的核心依赖）
4. 特效支持：发光、阴影、轮廓等

5. 更完善的排版工具

TMP Text 组件关键属性：

- Font Asset: 字体文件对应 TMP 资源
- Vertex Color: 顶点色（为什么叫 vertex color? 因为文字通过 Mesh 顶点 + 纹理渲染，颜色存储在顶点上）
- Spacing Options: 字间距/行间距/段间距
- Wrapping/Overflow: 溢出处理方式

3 在代码中访问 TMP 字符信息

[p.8-9]

引入 TMPPro 命名空间后可通过代码访问每个字符并修改。

TMP_Text 核心属性：

- text (string): 文字内容
- textInfo (TMP_TextInfo): 全部文字信息
- font (TMP_FontAsset): 字体资源
- textBounds (Bounds): 有效文本框大小

TMP_TextInfo 包含：

- characterInfo[]: 每个字符的详细信息（顶点位置、材质、UV 等）
- wordInfo[]: 每个单词的信息
- lineInfo[]: 每行的信息
- meshInfo[]: 渲染字符的 Mesh 数据——修改这些数据即可实现文字动画效果

4 编写文字动画系统

[p.11]

4.1 基础文字动画

修改 TMP_TextInfo 中的 meshInfo 来逐字符改变位置/旋转/缩放/颜色：

```

void AnimateText(TMP_Text textComponent) {
    textComponent.ForceMeshUpdate(); // 强制更新Mesh
    TMP_TextInfo textInfo = textComponent.textInfo;
    for (int i = 0; i < textInfo.characterCount; i++) {
        TMP_CharacterInfo charInfo = textInfo.characterInfo[i];
        if (!charInfo.isVisible) continue;

        int matIndex = charInfo.materialReferenceIndex;
        int vertexIndex = charInfo.vertexIndex;
        Vector3[] vertices = textInfo.meshInfo[matIndex].vertices;

        // 对4个顶点做变换（每个字符Quad有4个顶点）
        Vector3 offset = new Vector3(0, Mathf.Sin(Time.time + i) * 5, 0);
        for (int j = 0; j < 4; j++) {
            vertices[vertexIndex + j] += offset;
        }
    }
    // 将修改应用到渲染
    textComponent.UpdateVertexData(TMP_VertexDataUpdateFlags.Vertices);
}

```

4.2 打字机效果 (Typewriter)

控制 `maxVisibleCharacters` 属性逐步增加即可实现：

```

IEnumerator Typewriter(TMP_Text text, string content, float speed) {
    text.text = content;
    text.maxVisibleCharacters = 0;
    for (int i = 0; i <= content.Length; i++) {
        text.maxVisibleCharacters = i;
        yield return new WaitForSeconds(speed);
    }
}

```

打字机效果的变体：

- 匀速出现：固定间隔
- 加速/减速出现：使用动画曲线控制间隔
- 逐字弹出/弹入：同时修改每字符缩放（scale pop-in 动画）
- 随机抖动：每个字符出现时添加随机偏移

4.3 Inline 标记动画

模仿 TMP 的富文本标签（如 `<color=red>`），自定义标记来触发特定动画。例如 `<wave>` 文字 `</wave>` 使该段文字产生波浪动画。

实现思路：解析文本中的自定义标记 → 记录标记位置的字符索引 → 在 Update 中只对标记范围内的字符应用动画。

5 课后习题详解

[p.12]

5.1 题 1：每个字符逐渐缩小至消失

```
IEnumerator ShrinkToVanish(TMP_Text text, float duration) {
    text.ForceMeshUpdate();
    TMP_TextInfo info = text.textInfo;
    float elapsed = 0;

    while (elapsed < duration) {
        elapsed += Time.deltaTime;
        float progress = elapsed / duration;

        for (int i = 0; i < info.characterCount; i++) {
            TMP_CharacterInfo ci = info.characterInfo[i];
            if (!ci.isVisible) continue;
            int vi = ci.vertexIndex + ci.materialReferenceIndex * 0; // 简化处理
            Vector3[] verts = info.meshInfo[ci.materialReferenceIndex].vertices;

            // 计算字符中心，向中心缩放
            Vector3 center = (verts[vi]+verts[vi+1]+verts[vi+2]+verts[vi+3])/4;
            float scale = 1 - progress; // 从1到0
            for (int j = 0; j < 4; j++) {
                verts[vi+j] = center + (verts[vi+j] - center) * scale;
            }
        }
        text.UpdateVertexData(TMP_VertexDataUpdateFlags.Vertices);
        yield return null;
    }
}
```

5.2 题 2：为中文字体制作 TMP Font Asset

Window → TextMeshPro → Font Asset Creator:

1. Source Font File: 选择中文字体（如 SimSun/SimHei/.ttf 文件）
2. Character Set: 选择需要的字符集（Custom Characters 可手动输入常用汉字减少图集大小）
3. Atlas Resolution: 图集分辨率（中文字符多建议 2048×2048 或 4096×4096 ）
4. Generate Font Atlas → Save